TECNOLÓGICO NACIONAL DE MÉXICO
INSTITUTO TECNOLÓGICO DE TIJUANA

SUBDIRECCIÓN ACADÉMICA
DEPARTAMENTO DE SISTEMAS Y COMPUTACIÓN

SEMESTRE:
Enero-Junio 2026

CARRERA:
Ingeniería en Sistemas Computacionales

MATERIA:
Patrones de diseño

TÍTULO:
Actividad de evidencia prototype

UNIDAD:
2

NOMBRE Y NÚMERO DE CONTROL:
Kevin Omar Alvarez Hernandez 22210280

NOMBRE DEL MAESTRO:
Maribel Guerrero Luis

Índice

Índice.....2
Introducción.....3
Código implementado:..... 4
Resultados:..... 9
Conclusión:..... 10

Introducción

Prototype

¿Qué es?
Es un patrón creacional que permite crear nuevos objetos clonando instancias existentes (prototipos) en lugar de crearlos desde cero con `new`. Es ideal cuando la creación de un objeto es costosa, compleja o requiere muchos pasos, permitiendo copiar estructuras complejas y modificarlas sin afectar al original.

Características:
Clonación de objetos
Reducción de costos
Independencia de subclases
Estructura
Uso de prototipos prefabricados

Ventajas:
Creación eficiente de objetos
Reducción de la subclasificación
Creación dinámica en tiempo de ejecución
Desacoplamiento
Simplificación de la instanciación
Mayor flexibilidad

Ejemplos:
Gestión de documentos
Figuras geométricas
Células mitóticas
Medios de transporte

Código implementado:

```
using System;

namespace PatronPrototipo
{
    0 referencias
    class program
    {
        0 referencias
        static void Main(string[] args)
        {
            //1
            ExamenPrototype patrones = new PatronesPrototype();
            ExamenPrototype Lenguajes = new LenguajesPrototype();
            ExamenPrototype Simulacion = new SimulacionPrototype();
            ExamenPrototype Quimica = new QuimicaPrototype();
            ExamenPrototype Informatica = new InformaticaPrototype();
            ExamenPrototype Matematicas = new MatematicasPrototype();
            ExamenPrototype Circuitos = new CircuitosPrototype();
            ExamenPrototype Contabilidad = new ContabilidadPrototype();

            //2
            //
            ExamenPrototype ExamenPatrones = patrones.Clone();
            ExamenPatrones.Grupo = "Matutino";
            ExamenPatrones.Docente = "Gerardo";
            ExamenPatrones.Salon = "9301";
```

```
            //2
            //
            ExamenPrototype ExamenPatrones = patrones.Clone();
            ExamenPatrones.Grupo = "Matutino";
            ExamenPatrones.Docente = "Gerardo";
            ExamenPatrones.Salon = "9301";
            ExamenPatrones.Tipo = "Precencial";
            Console.WriteLine(ExamenPatrones.VerExamen());

            ExamenPrototype ExamenLenguajes = Lenguajes.Clone();
            ExamenLenguajes.Grupo = "Vespertino";
            ExamenLenguajes.Docente = "Jonathan";
            ExamenLenguajes.Salon = "9302";
            ExamenLenguajes.Tipo = "Precencial";
            Console.WriteLine(ExamenLenguajes.VerExamen());

            ExamenPrototype ExamenSimulacion = Simulacion.Clone();
            ExamenSimulacion.Grupo = "Matutino";
            ExamenSimulacion.Docente = "Jose";
            ExamenSimulacion.Salon = "9203";
            ExamenSimulacion.Tipo = "Virtual";
            Console.WriteLine(ExamenSimulacion.VerExamen());
```

```
42                  ExamenPrototype ExamenQuimica = Quimica.Clone();
43                  ExamenQuimica.Grupo = "Matutino";
44                  ExamenQuimica.Docente = "Katerine";
45                  ExamenQuimica.Salon = "501";
46                  ExamenQuimica.Tipo = "Virtual";
47                  Console.WriteLine(ExamenQuimica.VerExamen());
48
49                  ExamenPrototype ExamenInformatica = Informatica.Clone();
50                  ExamenInformatica.Grupo = "Verpertino";
51                  ExamenInformatica.Docente = "Susana";
52                  ExamenInformatica.Salon = "304";
53                  ExamenInformatica.Tipo = "Precencial";
54                  Console.WriteLine(ExamenInformatica.VerExamen());
55
56                  ExamenPrototype ExamenMatematicas = Matematicas.Clone();
57                  ExamenMatematicas.Grupo = "Matutino";
58                  ExamenMatematicas.Docente = "Abigail";
59                  ExamenMatematicas.Salon = "9401";
60                  ExamenMatematicas.Tipo = "Precencial";
61                  Console.WriteLine(ExamenMatematicas .VerExamen());
62
```

```
70                  ExamenPrototype ExamenContabilidad = Contabilidad.Clone();
71                  ExamenContabilidad.Grupo = "Vespertino";
72                  ExamenContabilidad.Docente = "Miguel";
73                  ExamenContabilidad.Salon = "706";
74                  ExamenContabilidad.Tipo = "Precencial";
75                  Console.WriteLine(ExamenContabilidad.VerExamen());
76
77                  Console.ReadKey();
78              }
79          }
80      }
```

```
using System;
using System.Collections.Generic;
using System.Text;

namespace PatronPrototipo
{
    7 referencias
    public abstract class ExamenPrototype
    {
        protected string _docente;
        protected string _grupo;
        protected string _salon;
        protected string _tipo;
        8 referencias
        public string Docente { set => _docente = value; }
        8 referencias
        public string Grupo { set => _grupo = value; }

        8 referencias
        public string Salon { set => _salon = value; }

        8 referencias
        public string Tipo { set => _tipo = value; }
        16 referencias
        public abstract ExamenPrototype Clone();
        16 referencias
        public abstract string VerExamen();
    }
}
```

```
using System;
using System.Collections.Generic;
using System.Text;

namespace PatronPrototipo
{
    2 referencias
    public class PatronesPrototype : ExamenPrototype
    {
        9 referencias
        public override ExamenPrototype Clone()
        {
            return (PatronesPrototype)this.MemberwiseClone();
        }
        9 referencias
        public override string VerExamen()
        {
            return $"docente {_docente} grupo {_grupo} salon {_salon} tipo {_tipo}";
        }
    }
}
```

```
using System;
using System.Collections.Generic;
using System.Text;

namespace PatronPrototipo
{
    2 referencias
    public class LenguajesPrototype : ExamenPrototype
    {
        9 referencias
        public override ExamenPrototype Clone()
        {
            return (LenguajesPrototype)this.MemberwiseClone();
        }
        9 referencias
        public override string VerExamen()
        {
            return $"docente {_docente} grupo {_grupo} salon {_salon} tipo {_tipo}";
        }
    }
}
```

```
using System;
using System.Collections.Generic;
using
    { } namespace System

namespace PatronPrototipo
{
    2 referencias
    public class SimulacionPrototype : ExamenPrototype
    {
        9 referencias
        public override ExamenPrototype Clone()
        {
            return (SimulacionPrototype)this.MemberwiseClone();
        }
        9 referencias
        public override string VerExamen()
        {
            return $"docente {_docente} grupo {_grupo} salon {_salon} tipo {_tipo}";
        }
    }
}
```

```
using System;
using System.Collections.Generic;
using System.Text;

namespace PatronPrototipo
{
    2 referencias
    public class QuimicaPrototype : ExamenPrototype
    {
        9 referencias
        public override ExamenPrototype Clone()
        {
            return (QuimicaPrototype)this.MemberwiseClone();
        }
        9 referencias
        public override string VerExamen()
        {
            return $"docente {_docente} grupo {_grupo} salon {_salon} tipo {_tipo}";
        }
    }
}
```

```
using System;
using System.Collections.Generic;
using System.Text;

namespace PatronPrototipo
{
    2 referencias
    public class InformaticaPrototype : ExamenPrototype
    {
        9 referencias
        public override ExamenPrototype Clone()
        {
            return (InformaticaPrototype)this.MemberwiseClone();
        }
        9 referencias
        public override string VerExamen()
        {
            return $"docente {_docente} grupo {_grupo} salon {_salon} tipo {_tipo}";
        }
    }
}
```

```
using System;
using System.Collections.Generic;
using System.Text;

namespace PatronPrototipo
{
    2 referencias
    public class MatematicasPrototype : ExamenPrototype
    {
        9 referencias
        public override ExamenPrototype Clone()
        {
            return (MatematicasPrototype)this.MemberwiseClone();
        }
        9 referencias
        public override string VerExamen()
        {
            return $"docente {_docente} grupo {_grupo} salon {_salon} tipo {_tipo}";
        }
    }
}
```

Resultados:

```
C:\Users\kevin\source\repos\

docente Gerardo grupo Matutino salon 9301 tipo Precencial
docente Jonathan grupo Vespertino salon 9302 tipo Precencial
docente Jose grupo Matutino salon 9203 tipo Virtual
docente Katerine grupo Matutino salon 501 tipo Virtual
docente Susana grupo Verpertino salon 304 tipo Precencial
docente Abigail grupo Matutino salon 9401 tipo Precencial
docente Armando grupo Matutino salon 9305 tipo Virtual
docente Miguel grupo Vespertino salon 706 tipo Precencial
```

Conclusión:

Este patrón es muy útil para la reutilización de la creación de clases, involucrando la herencia en una clase base que es la que inicializa los métodos básicos y datos protegidos; están las clases para que cada una no sea afectada por otra (encapsulamiento).